# What Structurally Breaks an Agent? Auditing Difficulty in Stateful Tool-Use Workflows

Anonymous Author(s)

## ABSTRACT

An aggregate task score does not distinguish failures of scheduling, state retention, and candidate selection. We develop a small executable instrument that varies dependency depth, candidate count, active state width, and access to current-state snapshots while holding the number of required operations fixed. Two dependency-graph families expose an important limitation of structural control: increasing depth at fixed work changes both scheduling freedom and the number of reachable completion states. We quantify these couplings instead of treating the controls as universally independent. The primary study contains 49,152 generated tasks and 393,216 reference-controller rollouts. A separately frozen follow-on design adds 295,936 rollouts covering operation count, retention strategy, snapshot cadence, hidden state resampling, recovery, query locality, update removal, candidate indexing, redundant computation, and relabeling. The ablations expose representation dependence: residue caches and precomputed plans solve tasks that defeat limited raw-value caches, while indexed selection removes the serial-scan branching deficit. Hidden resampling instead imposes an exact success ceiling that additional state tracking cannot overcome. Retry-enabled elimination changes the apparent failure threshold at an explicit tool-call cost. The contribution is an executable diagnostic instrument, analytic calibration, and an audit of its assumptions; a small, single-model local CPU study adds an exploratory model-in-the-loop check, but its 256 tasks yielded no completed workflows for the 0.5B model. This floor result does not show that the task dimensions have no effect and supports no cross-provider or reasoning-budget claim. A separate 16-episode, two-seed fractional factorial with one Copilot-routed model completed half the tasks; its small sample and aliased interactions make the contrasts descriptive rather than confirmatory.

## KEYWORDS

agent evaluation, task structure, dependency graphs, state retention, experimental design

## 1 INTRODUCTION

What makes a task difficult for an agent? A long sequence may require little planning if each next action is obvious. A short task may fail because a necessary value is no longer visible. Many available actions need not be problematic when their preconditions are easy to evaluate. These cases have different remedies, yet a single completion score treats them alike.

Existing work already establishes that task complexity matters. DyVal generates reasoning problems with controllable graph structure [1]; controlled puzzles examine performance as compositional complexity increases [2]; and task depth and capability breadth have been studied in multi-agent evaluation [15]. More directly, TRAJECT-Bench varies tool-trajectory breadth and depth and reports trajectory-level selection, argument, and ordering metrics [10]. The Reward Hacking Benchmark studies independent and chained multi-step tool tasks across frontier models, including harder task variants [11]. SynthAPI varies tool, task, and database diversity in synthetic executable environments, although its question is training-data scaling rather than capability diagnosis [12]. Synthetic POMDPs have also been designed with explicit, theoretically characterized memory-demand structures while preserving transition invariants [14]. Our remaining question is narrower: can a small tool-execution workflow isolate state-retention and feedback conditions while preserving required work and auditing unavoidable structural couplings?

The scope of novelty is necessarily narrow. Controllable graph difficulty, reasoning-budget comparisons, tool-trajectory diagnostics, multi-step tool tasks, and memory benchmarks all have substantial prior work [1, 2, 4–7, 9–13]. Our candidate distinction is the combination of a fixed-work, state-mutating workflow with explicit contrasts between state snapshots and history-reconstructible feedback, plus audits of how controller representation changes the resulting profile. This is a testable distinction, not a novelty finding: neither the reference controllers nor the two small, single-model evaluations validate a general capability-profile contribution, and no direct benchmark comparison has been performed.

We study four operational controls: the longest prerequisite chain, the number of candidates within an operation, the number of active state registers, and whether current register values are repeated in feedback. The total number of required operations is always eight. This is a *fixed-work depth intervention*: depth changes precedence, not episode length. Candidate count is likewise local to an operation, not the number of globally available actions. These distinctions are necessary for interpreting the experiment.

The present study is a methods and calibration paper. All evaluated controllers are short, inspectable programs with deliberately restricted resources. Their failures are not surprising discoveries about general reasoning. Instead, they are positive and negative controls: an instrument that cannot distinguish a known retention deficit from a known scheduling deficit should not be used to diagnose less interpretable systems. We now also report exploratory evidence from one small local model, a small Copilot-routed pilot, and a two-seed Copilot fractional factorial. None supports general capability claims.

Our contributions are an executable fixed-work factorial task family; an exact audit showing which structural metrics remain fixed and which necessarily move together; and a reference-controller evaluation with paired uncertainty estimates and analytic success predictions, followed by controlled ablations of the assumptions underlying those profiles. We also report one local-model factorial, paired pilot traces, and a small fractional

factorial from one Copilot-routed GPT model. We make no claim to introduce controlled complexity evaluation itself, to establish a universal failure threshold, or to estimate cross-model effects from the pilot.

The distinction between task structure and controller representation is especially important. Increasing the number of candidate slots makes serial examination expensive, but not an indexed lookup. Increasing the number of registers creates more values to retain, but a known deterministic workflow can be compiled into a list of future answers. Conversely, newly sampled unobserved values cannot be recovered by a better retention policy. A useful structural profile must say which of these cases its controls measure.

## 2 RELATED WORK

*Controlled complexity, memory demand, and tool trajectories.* DyVal uses directed acyclic graphs to generate reasoning tasks at adjustable complexities [1]. Shojaee et al. use controlled puzzle environments to study complexity-dependent performance and computation, including reasoning effort under fixed inference budgets [2]. MorphoBench adapts question difficulty to model capability and evaluates GPT and Gemini models [3]. These works establish that controllable difficulty and model-based scaling studies are not new contributions by themselves. TRAJECT-Bench varies parallel breadth and sequential depth in executable, production-style tool trajectories and analyzes tool selection, argument correctness, and ordering [10]. RHB evaluates chained tool workflows across 13 models and tests harder variants and environmental interventions [11]. SynthAPI uses executable stateful environments and independently controls tool, task, and database diversity to study training-data composition across hundreds of thousands of tasks [12]. Their objectives differ from ours, but their scope makes broad claims about the first controlled or structural tool-use benchmark untenable. Wang et al. provide an especially relevant precedent for controllable memory difficulty: their synthetic POMDPs define memory-demand structures and construct tunable environments while preserving the belief MDP [14]. Our snapshot/history contrast is not a partial-observability intervention, because successful action/update history is sufficient to reconstruct the current registers. It measures the burden of maintaining state internally rather than a loss of information. The hidden-resampling extension introduces missing information, but is currently tested only with programmed policies. Tang et al. analyze depth and width in multi-agent systems [15]; their width denotes capability diversity, whereas ours counts mutable registers. Our narrower candidate contribution is an audited, stateful execution design that separates retention, candidate selection, and feedback access, while measuring how controller representation changes the resulting profile. The current experiments validate that design with programmed controllers; they do not establish the profile of a language model.

*Diagnostic environments.* The Behaviour Suite explicitly evaluates core capabilities through carefully designed tasks [16]. BabyAI provides synthetic instruction-following levels of increasing difficulty and a heuristic expert [17]. Our reference-policy design follows this diagnostic tradition. The workflow setting substitutes dependency-constrained operations for movement and makes the required work and failure rules directly inspectable. We have not experimentally compared our instrument with either platform.

*Interactive and long-horizon evaluation.* $\tau$-bench evaluates tool-agent-user interaction using final database states [18]. Kwa et al. measure the human-duration horizons of software tasks [19]. Those evaluation targets differ from a fixed-work intervention on precedence. ARIES studies policies that transform interactive thought graphs [20]; it does not provide evidence for independence of the controls used here. The partial-observation literature distinguishes current observations from history-conditioned information [21]. That distinction is central to our observation intervention: omitted snapshots do not destroy information already recoverable from history.

*Tool-use benchmarks and nearest overlap.* API-Bank, ToolBench, and AppWorld evaluate tool-augmented models on runnable or interactive tasks, while AgentBench spans multiple interactive environments [5–8]. MCPAgentBench is a particularly close recent comparison: it evaluates multi-step tool invocation and presents candidate tool lists with distractors [9]. TRAJECT-Bench is a closer overlap for controlled trajectory structure and intermediate failure analysis; RHB is a closer overlap for multi-step tool-use difficulty and model-based experiments [10, 11]. MemCalib directly benchmarks model use of retrieved memory, although it measures the influence of natural-language propositions on responses rather than retention of mutable workflow state [13]. Accordingly, neither multi-step tool use, trajectory depth, candidate selection, nor memory evaluation is claimed as new here. Our candidate distinction is the fixed-work state-mutating intervention and its explicit separation of visible snapshots from history-reconstructible state. We have not reimplemented or directly compared against these benchmark suites, and the programmed-controller and single-model results cannot establish that this distinction improves evaluation of language models.

*Reasoning-budget evaluation.* Reasoning effort and inference compute have already been studied as experimental variables, including budget-matched comparisons of reasoning strategies [4]. The Illusion of Thinking also studies complexity-dependent reasoning effort [2]. The open empirical question for this project is not whether budget matters in general, but whether budget changes model performance differently across audited state-retention, candidate-selection, and feedback interventions. That interaction has not been measured in this manuscript.

*Workshop precedents.* We inspected four published ALA 2025 papers as methodological precedents, not as a systematic review or an empirical comparison. Lipkis and Agogino study failure-probability estimation and the difficulty of sampling rare outcomes [22]. Our exact small-task probabilities serve a different purpose: they expose when an empirical zero is a sampling limitation. Dynamic Thinker optimizes decision-time search with compute-dependent objectives [23]. Our comparison counts explicit local checks and preparation work; it does not reproduce that learned metareasoning experiment. Nourzad et al. compare a task-graph scheduling architecture with heuristic and random baselines using makespan and scheduling time [24]. Our scheduler

checks ordering validity, not assignment to heterogeneous processors or makespan optimization. Leal et al. vary deliberation cost in option-based learning [25]; our retry allowance changes execution opportunities, not option termination or a training objective.

These precedents motivate reporting the concrete intervention, the unrestricted reference, the resource charged, and the uncertainty of the outcome. They do not establish that our task family is novel. The closest methodological overlap remains controlled graph generation and diagnostic environments; the claimed contribution is the combined audit of stateful execution, representation-sensitive profiles, and recoverability.

# 3 EXECUTABLE TASK DESIGN

## 3.1 State, actions, and feedback

A task consists of a prerequisite graph $G = (V, E)$ with $|V| = N = 8$ operations, $w \in \{1, 2, 4, 8\}$ active registers, and a candidate count $b \in \{2, 4, 8\}$. Registers take values in $\mathbb{Z}_8$. The initial eight-slot array is sampled uniformly and shown once. Slots outside the active width are unused padding. Each operation $v$ specifies a query register $q_v$, an update register $u_v$, a nonzero increment $\delta_v \in \{1, \ldots, 7\}$, and a random permutation $\sigma_v$ of the codes $0, \ldots, b-1$.

The sole execution tool accepts an operation identifier and candidate slot:

$$\texttt{execute}(v, a).$$

The action succeeds exactly when all predecessors are complete and

$$\sigma_v(a) = x_{q_v} \bmod b.$$

It then marks $v$ complete and applies $x_{u_v} \leftarrow (x_{u_v} + \delta_v) \bmod 8$. Each active register is queried exactly $N/w$ times; query positions are shuffled. Updates are sampled independently and need not cover every register. Node labels, initial values, update draws, and candidate-order draws are seeded.

An invalid dependency or candidate terminates the episode immediately with an explicit failure category. Invalid tool arguments raise an error. The goal is to complete all eight operations. Thus every successful episode uses exactly eight execution calls. There is no reset, inspection tool, retry, reward shaping, or recovery allowance.

Both feedback conditions reveal completed operations and the applied register update. The *snapshot* condition also repeats all current register values. The *history* condition replaces the eight values with null entries after the initial observation. Since the initial state and all successful updates are available, an exact history-tracking controller can reconstruct every value in either condition. The intervention removes convenient refreshes, not recoverable information. It therefore cannot establish robustness to unobserved stochastic changes.

## 3.2 Two fixed-work graph families

Depth $d \in \{1, 2, 4, 8\}$ is the longest prerequisite chain, measured in nodes. Let $k = N/d$. The *chains* family has $k$ disjoint chains of length $d$. The *barriers* family has $d$ layers of $k$ operations; every operation in a layer depends on every operation in the preceding layer. The two families coincide at $d = 1$ and $d = N$ but differ at intermediate depths. They are a graph-topology robustness check, not two independent domain generators: the state and candidate mechanics are shared.

| Family | $d$ | Edges | Masks | Orders | Ready nodes |
|---|---|---|---|---|---|
| Chains | 1 | 0 | 256 | 40320 | 4.02 |
| Chains | 2 | 4 | 81 | 2520 | 2.70 |
| Chains | 4 | 6 | 25 | 70 | 1.67 |
| Chains | 8 | 7 | 9 | 1 | 1.00 |
| Barriers | 1 | 0 | 256 | 40320 | 4.02 |
| Barriers | 2 | 16 | 31 | 576 | 2.13 |
| Barriers | 4 | 12 | 13 | 16 | 1.33 |
| Barriers | 8 | 7 | 9 | 1 | 1.00 |

**Table 1: Exact structural audit at $N = 8$. "Masks" counts reachable sets of completed operations; "orders" counts successful operation permutations. Ready nodes are averaged uniformly over nonterminal reachable completion masks, not over a controller's trajectories.**

Increasing depth changes precedence while preserving $N$, the state alphabet, candidate count, active width, and optimal execution-call count. Holding the seed fixed pairs node labels, register draws, and candidate orders across the snapshot intervention and, at fixed depth, across graph families. Across width and candidate levels, the same random draws are reused before level-specific projections. This reduces avoidable surface variation without asserting identical semantics across levels.

## 3.3 What is and is not controlled

We enumerate reachable completion masks and count valid schedules by dynamic programming. For chains, the mask count is $(d+1)^k$; for barriers, it is $d(2^k - 1) + 1$. The numbers of valid schedules are respectively

$$L_{\text{chains}} = \frac{N!}{(d!)^k}, \qquad L_{\text{barriers}} = (k!)^d.$$

Table 1 shows that depth necessarily changes reachable completion sets and scheduling freedom. Increasing local candidate count also multiplies the number of exposed slots across ready operations. These are not four orthogonal graph metrics.

For a fixed initial state, commuting additive updates make the register state a deterministic function of the completion mask. The mask audit is therefore exact for successful executions of an instance, but excludes failure sinks and does not claim to count a Cartesian product of all register assignments. Across the task distribution, active initial-state entropy is $3w$ bits; the entropy of the code tested by a candidate is $\log_2 b$ bits. Width and candidate count consequently interact in decision-relevant information even though the register alphabet is fixed.

The construction preserves array-slot count and the single tool interface. It does not preserve serialized prompt length, token count, edge count, global action branching, or total state-space size. No prompt tokens are used by the local controllers. A causal statement in this study concerns an operational intervention inside

this generator, including its documented couplings; it cannot be interpreted as an isolated effect of an abstract complexity measure.

# 4 REFERENCE CONTROLLERS AND CALIBRATION

All controllers consume public task rules and returned observations. None can read simulator state. Except for the random scheduler, each chooses the smallest-labelled ready operation.

The *Exact* controller retains all active values and examines candidates until it finds the matching code. *Cache-2* and *Cache-4* retain only the first two or four active register values and apply public updates to that fixed cache. A visible snapshot may answer the current query even outside the cache. A missing value causes a uniform candidate guess. These controllers intentionally do not infer forgotten values from their successful guesses or precompute future answers.

*Scan-1* and *Scan-4* retain all registers but examine at most one or four candidate slots. If none matches, they choose uniformly among unexamined slots. *Cache-2 + Scan-4* combines the two restrictions. The *Random schedule* controller retains all values and selects uniformly among incomplete operations without checking dependencies. The *Uniform candidate* controller schedules correctly but guesses every candidate. These are resource-restricted algorithms, not trained agents or models of human cognition.

*Analytic predictions.* For a known queried value and a uniformly permuted candidate list, a controller checking $s$ candidates has average per-action success

$$p_s = \min\{1, (s+1)/b\}.$$

The extra one comes from the final guess. More precisely, conditional on an instance's actual state and candidate order, success is one when the correct slot is among those examined and $1/(b-s)$ otherwise. A query outside the cache without a snapshot has probability $1/b$. We multiply these instance-conditional probabilities along the deterministic ready schedule. The initial observation is visible in both conditions, so the first action is never incorrectly counted as a forgotten-state query. State along a successful path is deterministic, and new uniform guesses are independent, making this calculation exact for the specified policy.

Uniform candidate success is $b^{-N}$. The random scheduler samples a uniform operation permutation, yielding

$$P_{\text{chains}} = (d!)^{-k}, \qquad P_{\text{barriers}} = \frac{(k!)^d}{N!}.$$

These formulas are calibration targets rather than fitted explanations. In particular, critical-path depth alone cannot determine success even for this simple scheduler: at $N = 8, d = 2$, its probability is $1/16$ for chains and $1/70$ for barriers.

# 5 EXPERIMENTAL PROTOCOL

The full design has $2 \times 4 \times 3 \times 4 \times 2 = 192$ cells. Each uses seeds $0, \ldots, 255$ and all eight controllers, for 49,152 tasks and 393,216 episodes. The same seed is a paired block across levels and policies. Repeated conditions are not treated as independent trials. The configuration and analysis rules were written before the primary run in a local protocol file; this is not an external preregistration.

The primary outcome is complete task success. We also record completed operations, execution calls, candidate evaluations, and first failure type. There are no exclusions. An Exact-controller failure aborts the run. Tests verify the full grid over additional seeds, match graph schedule counts against exhaustive small-graph enumeration, check observation masking and public updates, and exercise explicit invalid-action behavior.

Within each family, a main contrast is the high-minus-low endpoint success-rate difference, averaged equally over every other level. We report all four contrasts and a specified width-by-observation difference-in-differences:

$$I_{w,o} = [S(8, \text{history}) - S(1, \text{history})] - [S(8, \text{snapshot}) - S(1, \text{snapshot})].$$

The remaining controls are averaged equally in each $S$. Percentile bootstrap bands use 2,000 resamples of the 256 paired seed blocks, with a fixed analysis seed. They are descriptive 95% bands, not simultaneous significance statements. All contrast intervals and secondary outcomes are included in the supplementary artifact.

## 5.1 Separately frozen follow-on design

After inspecting the primary results, we specified seven ablation grids containing 1,156 task–controller configurations (Table 2). Each uses seeds 10,000, . . . , 10,255, disjoint from the primary seeds, giving 295,936 additional episodes and 689,152 episodes overall. A configuration includes its controller; the count is not a count of distinct task environments. Some anchors deliberately recur across grids, and their repetitions do not create independent evidence.

The same seed is paired across each explicitly stated comparison. Changing $N$ necessarily changes the number of generated operations and random draws; this is common-seed pairing, not identical task content. Follow-on contrasts average only their stated anchors, not the complete primary grid. All 77 selected contrasts are stored, including negative and zero differences. The follow-on protocol was frozen locally before these runs, but after observing the first study; neither protocol is an external preregistration.

We additionally compute every endpoint interaction of orders two through four in the original factorial data: six pairwise, four three-way, and one four-way interaction for each controller and family, or 176 estimates. These are exploratory follow-on analyses. They use signed differences of endpoint cell means, averaging all unaffected controls equally. Bands again resample paired seed blocks 2,000 times, with no multiplicity-adjusted or causal-mechanism significance claims.

# 6 RESULTS

## 6.1 Distinct structural profiles

Exact succeeds on every generated task. Cache-2 averages 75.57% success on chains and 75.63% on barriers; Cache-4 averages 87.87% and 87.89%. The observed Scan-1 success rates are 33.46% and 33.52%, whereas Scan-4 reaches 67.43% and 67.38% (Table 3). These averages reflect the chosen equal-weight grid, not a natural task distribution.

The largest depth contrast belongs to the random scheduler: −100 percentage points from depth one to eight in each family. The largest candidate contrast belongs to Scan-1, also −100 points.

| Grid | Intervention and anchors | Configurations |
|---|---|---|
| Work | $N = 4, 8, 16$; $d = 2, 4$; $w = 4$; initial-only or every-step snapshots | 192 |
| Memory | $N = 16$, $d = 4$; fixed, next-use, residue, plan, and stale retention; initial-only or every-fourth snapshots | 96 |
| Observation | $N = 8$, $d = 4$; periods 0, 1, 2, 4 crossed with public updates or hidden independent resampling | 256 |
| Recovery | $N = 8$, $d = 4$; error allowances 0, 2, 8, 56; trials with replacement or elimination; both snapshot endpoints | 256 |
| Locality | $N = 16$, $d = 4$; balanced, clustered, or concentrated queries crossed with additive or static state | 192 |
| Selection | $N = 8$, $d = 4$, $w = 4$; $b = 2, 4, 8$; serial, fixed, redundant, and indexed selection; every-step snapshots | 36 |
| Labels | $N = 8$, $d = 4$; original random identifiers versus reversed identifiers; both snapshot endpoints | 128 |

**Table 2: Follow-on design. Both graph families appear in every grid. Unless stated otherwise, $b = 4, 8$ and $w = 4, 8$. Grid-specific controller sets and the complete configuration manifest are supplied with the artifact.**

| Family | Controller | Success (%) | 95% interval | Predicted (%) |
|---|---|---|---|---|
| Barriers | Exact | 100.00 | [100.00, 100.00] | 100.00 |
| Barriers | Cache-2 | 75.63 | [75.51, 75.75] | 75.56 |
| Barriers | Cache-2 + Scan-4 | 51.06 | [50.87, 51.27] | 51.13 |
| Barriers | Cache-4 | 87.89 | [87.80, 88.00] | 87.93 |
| Barriers | Random schedule | 25.00 | [25.00, 25.00] | 25.37 |
| Barriers | Scan-1 | 33.52 | [33.45, 33.61] | 33.46 |
| Barriers | Scan-4 | 67.38 | [67.20, 67.56] | 67.42 |
| Barriers | Uniform candidate | 0.11 | [0.06, 0.18] | 0.13 |
| Chains | Exact | 100.00 | [100.00, 100.00] | 100.00 |
| Chains | Cache-2 | 75.57 | [75.46, 75.68] | 75.56 |
| Chains | Cache-2 + Scan-4 | 51.18 | [51.00, 51.38] | 51.18 |
| Chains | Cache-4 | 87.87 | [87.78, 87.96] | 87.93 |
| Chains | Random schedule | 25.98 | [25.39, 26.66] | 26.61 |
| Chains | Scan-1 | 33.46 | [33.40, 33.54] | 33.45 |
| Chains | Scan-4 | 67.43 | [67.26, 67.64] | 67.49 |
| Chains | Uniform candidate | 0.14 | [0.07, 0.21] | 0.13 |

**Table 3: Measured and instance-conditional predicted success. Intervals are paired seed-bootstrap bands. Degenerate bootstrap bands do not bound unseen rare events; Table 5 provides nondegenerate binomial intervals for the sparse scheduling condition.**

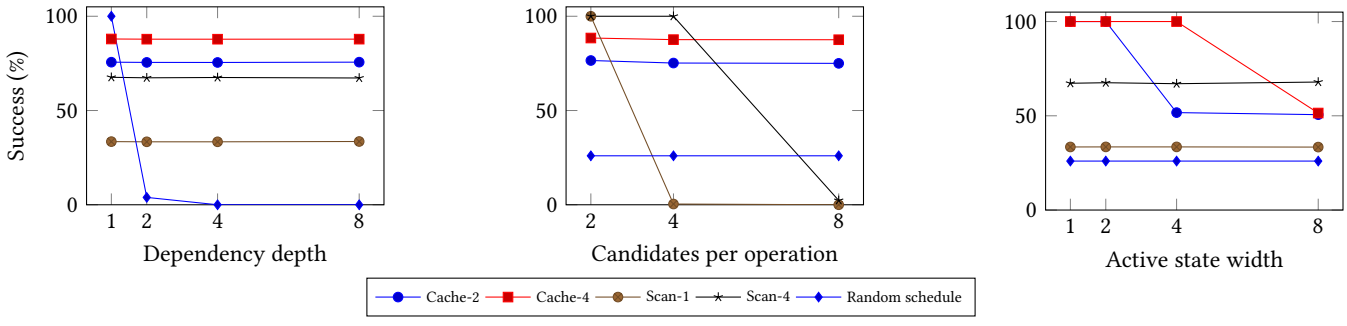



**Figure 1: Marginal profiles in the chains family, averaging all other controls equally. Points are simulation results, not fitted curves. The width panel averages snapshot and history conditions, so its roughly 50% cache floor hides the near-complete failure of the history-only condition at width eight. Full curves for both families are supplied.**

Cache-2 instead loses approximately 49 points as width rises from one to eight. Depth has negligible effects on the correctly scheduled cache and scan controls compared with their resource-specific

interventions. Small residual differences reflect instance permutations and finite simulation rather than a general equivalence claim (Table 4).

### 6.2 Observation refreshes and candidate checks differ

Both cache controllers succeed on every snapshot task, regardless of width. Without refreshes, Cache-2 succeeds on 51.15% of chains tasks averaged across widths, but only about 1.2% at width eight averaged over depth and candidate count. Its width-by-observation interaction is -98.80 percentage points in chains, versus −98.86 in barriers. The large interaction is expected: width creates no retention deficit when the current value is directly visible. Averaging observation conditions conceals this failure mode.

Increasing the scan limit changes the candidate profile but does not recover a forgotten value. With four candidates, Scan-4 always succeeds, while Scan-1 succeeds on fewer than 1% of tasks. With eight candidates, Scan-4 succeeds on 2.29% of chains tasks and 2.15% of barriers tasks, consistent with $(5/8)^8 \approx 2.33\%$ averaged over candidate-order draws. The combined Cache-2 + Scan-4 controller remains sensitive to both width and candidate count. Additional local checks and increased retained state are therefore non-interchangeable resources in this instrument.

This is not an experiment on reasoning-token budgets. Scan limits its count predicate evaluations, not tokens or elapsed computation; early failures also shorten episodes. The follow-on redundant-check control matches checks per decision, not total episode computation, and supports no inference-budget conclusion.

### 6.3 Calibration and rare events

The maximum absolute discrepancy between an overall observed rate and its analytic prediction is 0.63 percentage points, for the chains random scheduler. Agreement is a useful implementation check, not an independent validation of external relevance.

At barrier depth two, zero successes were observed although the exact probability is $1/70$. With 256 independent seeded instances and scheduling draws, this outcome has probability $(69/70)^{256} \approx 2.5\%$. The aggregate seed bootstrap then reports a degenerate 25% band because depth-one tasks always succeed and other sampled depths never do. It cannot resolve an unseen rare event. The Wilson interval for the depth-two cell extends to approximately 1.48%, including the exact 1.43% probability (Table 5). We therefore do not interpret a zero empirical rate or a zero-width bootstrap band as proof of impossibility. The Wilson intervals are a supplementary diagnostic added after observing the sparse scheduling outcomes, not a prespecified primary analysis.

### 6.4 Interactions beyond width and snapshots

The endpoint profiles are not additive. For the combined Cache-2 + Scan-4 controller, the candidate-by-width interaction is 47.90 percentage points in chains and 48.14 in barriers; the candidate-by-width-by-observation interaction reaches 95.02 and 95.90 points. These large signed differences arise partly from ceiling and floor effects: restoring a resource can matter little while another resource already prevents completion. They are not evidence for a universal three-factor law. The full pairwise and higher-order tables are in the appendix, and every interval is in the accompanying data.

## 7 ABLATIONS OF THE DIAGNOSTIC ASSUMPTIONS

### 7.1 Work is not fixed-work depth

The work grid increases $N$ from four to sixteen while retaining $d = 2, 4$, $w = 4$, and $b = 4, 8$. Exact succeeds throughout. Without repeated snapshots, Cache-2 falls from 13.43% to zero observed success: a paired difference of −13.43 points with a descriptive band $[-16.02, -10.94]$. With snapshots every step, the same cache succeeds throughout. The scan-limited controller instead falls from 56.59% to 50.05% under either snapshot endpoint. The aggregate floor near 50% is an artifact of averaging four-candidate tasks, which Scan-4 always solves, with eight-candidate tasks, whose errors compound.

This is a work intervention, not a claim that dependency depth and episode length are equivalent. More operations also create more updates and repeated queries. The graph changes because its layer width is $N/d$; any interpretation of the work contrast includes those effects. The original fixed-work depth experiment and this follow-on experiment answer different questions.

### 7.2 Retention strategy and preparation

We compare fixed caches with a *next-use* cache that retains at most two registers, prioritizing their known upcoming queries. At each snapshot, it selects future-use registers rather than the lowest indices; a successful guess also confirms the queried value modulo $b$. Public updates adjust the retained residues. Its scheduling information is public, but this is a stronger retention strategy than the deliberately non-adaptive fixed cache.

At $N = 16$ with no intermediate snapshots, both limited-cache strategies remain near the floor: Cache-2 has zero observed successes, and Next-use-2 has 0.05%. With a snapshot every fourth successful operation, Next-use-2 reaches 9.52%, compared with zero observed success for Cache-2. The paired difference is 9.52 points $[7.86, 11.18]$. Thus a register-count profile depends on what the controller retains and how refreshes are used; the result does not make the two-register cache generally adequate.

Two unrestricted representation controls solve every deterministic memory task. *Residue-8* tracks each active value modulo $b$, using $w \log_2 b$ value bits rather than $3w$. This follows because $b$ divides eight: updates modulo eight preserve the residue needed by the candidate predicate. *Precomputed plan* uses the visible initial state and public deterministic updates to compute all $N$ future candidate slots before execution. It then performs $N$ plan lookups without an execution-time register cache. The log separately records $N$ plan slots and $Nb$ candidate-table preparation units. This is neither zero memory nor free computation: initial planning uses state workspace and the plan itself stores information.

| Family | Controller | Depth | Candidates | Width | History |
|---|---|---|---|---|---|
| Barriers | Exact | 0.00 | 0.00 | 0.00 | 0.00 |
| Barriers | Cache-2 | 0.02 | -1.68 | -49.43 | -48.75 |
| Barriers | Cache-2 + Scan-4 | -0.10 | -74.79 | -33.32 | -32.63 |
| Barriers | Cache-4 | -0.08 | -0.89 | -48.42 | -24.21 |
| Barriers | Random schedule | -100.00 | 0.00 | 0.00 | 0.00 |
| Barriers | Scan-1 | 0.07 | -100.00 | -0.23 | 0.00 |
| Barriers | Scan-4 | -0.36 | -97.85 | 0.36 | 0.00 |
| Barriers | Uniform candidate | 0.13 | -0.34 | 0.13 | 0.00 |
| Chains | Exact | 0.00 | 0.00 | 0.00 | 0.00 |
| Chains | Cache-2 | 0.02 | -1.51 | -49.40 | -48.85 |
| Chains | Cache-2 + Scan-4 | -0.10 | -74.49 | -33.32 | -32.69 |
| Chains | Cache-4 | -0.08 | -0.90 | -48.54 | -24.27 |
| Chains | Random schedule | -100.00 | 0.00 | 0.00 | 0.00 |
| Chains | Scan-1 | 0.07 | -100.00 | -0.07 | 0.00 |
| Chains | Scan-4 | -0.36 | -97.71 | 0.65 | 0.00 |
| Chains | Uniform candidate | 0.13 | -0.42 | 0.00 | 0.00 |

**Table 4: High-minus-low main contrasts in percentage points. Depth: 8 minus 1; candidates: 8 minus 2; width: 8 minus 1; history: history minus snapshot. Each averages all remaining levels equally within its family. The supplement reports the corresponding paired intervals.**

| Family | $d$ | Observed (%) | Wilson interval | Exact (%) |
|---|---|---|---|---|
| Chains | 1 | 100.00 | [98.52, 100.00] | 100.0000 |
| Chains | 2 | 3.91 | [2.14, 7.04] | 6.2500 |
| Chains | 4 | 0.00 | [0.00, 1.48] | 0.1736 |
| Chains | 8 | 0.00 | [0.00, 1.48] | 0.0025 |
| Barriers | 1 | 100.00 | [98.52, 100.00] | 100.0000 |
| Barriers | 2 | 0.00 | [0.00, 1.48] | 1.4286 |
| Barriers | 4 | 0.00 | [0.00, 1.48] | 0.0397 |
| Barriers | 8 | 0.00 | [0.00, 1.48] | 0.0025 |

**Table 5: Random-scheduler calibration using one representative cell per depth and family, with 256 distinct seeds. Other candidate, width, and observation conditions repeat the same scheduling draws and do not increase this effective trial count. Wilson 95% binomial intervals retain uncertainty at zero observed successes.**

## 7.3 Snapshot cadence versus missing information

Snapshot periods count successful operations, not tool attempts. Period zero means the initial observation only; period one repeats state after every success. With public deterministic updates, Exact remains at 100% for periods zero, one, two, and four. Cache-2 reaches 0.29%, 100%, 4.64%, and 0.44%, respectively, on the $N = 8$ anchors. The difference is therefore a retention and refresh effect, not a lack of history-conditioned information.

We then independently resample all active registers uniformly after each successful operation. Feedback announces that resampling occurred but does not reveal new values unless a snapshot is due. Controllers invalidate stale cached values. Under initial-only feedback, Exact now has zero observed successes, despite its perfect tracking in the public-update condition. With every-step snapshots it returns to 100%; with period two it achieves 0.29%, and period four produces zero observed successes.

This case has an information bound, not merely an algorithm-specific failure rate. Without retries, let $v = \lceil N/h \rceil$ be the number of visible decisions at period $h > 0$, and let $v = 1$ for initial-only feedback. Any history-based controller has

$$P(\text{complete}) \leq b^{-(N-v)}.$$

At each masked decision the newly resampled target code is uniform, independent of the entire visible history. An otherwise exact controller attains the bound by selecting correctly at visible steps and guessing at masked steps. For $N = 8$, initial-only feedback gives $4^{-7}$ or $8^{-7}$, not literal impossibility. The aggregate analytic probability across the two candidate counts is approximately 0.0031%, explaining why a modest seed budget is unlikely to observe success.

## 7.4 Recovery changes the experiment

In the recovery grid, an invalid candidate returns its failure category without changing state or marking an operation complete. The episode ends only when its global error allowance $R$ is exceeded. Successful operations still apply updates. The maximum execution horizon is $N+R$ calls. We compare independent uniform guesses with candidate elimination, which remembers failed codes for the current operation. This extra memory and failure feedback must not be mistaken for additional internal reasoning.

With eight allowed errors and initial-only snapshots, uniform elimination achieves 6.54% success versus 1.27% for replacement: a paired gain of 5.27 points [4.15, 6.49]. Cache-2 with elimination reaches 44.63%. At $R = 56$, elimination solves every task, while replacement reaches 78.56%. The average elimination cost is 27.87 calls, compared with eight for an unrestricted exact controller.

The success of elimination at $R = N(b-1)$ is guaranteed, not an empirical discovery: no operation can require more than $b-1$ incorrect trials. For uniform replacement, the number of failed guesses before $N$ successes is negative binomial. For elimination, failures per operation are uniform on $\{0, \ldots, b-1\}$; convolving those counts gives the exact finite-budget probability. The appendix states both formulas. Retry budget, error feedback, and elimination memory together define a different task protocol from immediate termination.

### 7.5 Locality and removal of state updates

Balanced and clustered query schedules have the same active width and query counts. Clustering makes consecutive queries use the same register along the deterministic ready schedule. At $N = 16$ with public updates, Next-use-2 improves from 0.05% under balanced queries to 1.86% under clustered queries: a paired difference of 1.81 points $[1.22, 2.44]$. Reusing a confirmed residue can therefore help, but the controller still fails most tasks. Fixed Cache-2 has zero observed complete successes in both conditions; its progress and call counts nevertheless differ. A completion-only score misses that difference.

Concentrating every query on register zero yields 100% for both limited cache strategies. This is a deliberate *invalid width control*: the declared width remains four or eight, but the effective queried width is one. We report that audit failure instead of presenting concentration as a fair improvement at fixed memory demand.

The *Stale-8* controller retains initial values but ignores updates. Removing updates restores it from zero observed success on balanced additive tasks to 100% on balanced static tasks; unrestricted tracking remains at 100% in both. This negative control confirms that the update mechanism matters. Static tasks are easier for this stale strategy, not evidence that tracking mutable state is an intrinsic universal difficulty.

### 7.6 Candidate indexing and redundant checks

The *Fixed-check-4* controller evaluates four candidates per decision and uses every result. The *Sham-check-4* controller also evaluates four, but only the first result is allowed to affect selection; the other three are redundant. With four candidates, Fixed-check-4 succeeds at 100% and Sham-check-4 at 0.59%. With eight candidates, they reach 1.17% and zero observed success. Sham-check-4 produces exactly the same action traces as Scan-1 under paired seeds, despite its higher comparison cost. Useful computation, rather than charged computation alone, explains the difference.

*Indexed* constructs code-to-slot maps before execution and retains all required state. It succeeds at 100% for all tested candidate counts, using eight execution lookups and $Nb$ preparation entries (Table 6). Branching is therefore costly for the serial-scan controller but not an inherent obstacle to this representation. Index preparation grows with $b$, and we do not compare total wall time, energy, or token costs. The experiment is a counterexample to representation-free interpretation of a branching profile, not proof that indexing is costless.

### 7.7 Identifiers, floors, and aggregation

Reversing the randomly assigned operation identifiers preserves the graph audit and Exact's 100% success. It changes the deterministic ready order and hence some queried states. Scan-4's paired difference is $-0.24$ points $[-0.59, 0.15]$ with initial-only feedback, and Cache-2's is $-0.15$ $[-0.49, 0.15]$. This limited relabeling check does not establish invariance to arbitrary wording, prompts, or tool names. Random scheduling remains too sparse at the selected depth to resolve subtle differences.

Several ablations are dominated by ceiling and floor effects. We retain zero-result conditions, exact probabilities where derived, and secondary progress and resource outcomes rather than regenerate tasks after observing success. The follow-on grid is consequently useful for checking assumptions but not calibrated for ranking unfamiliar agents. Aggregate percentages must always be accompanied by the conditions averaged into them.

## 8 INTERPRETATION AND LIMITATIONS

The experiment shows that the instrument responds selectively to limitations introduced by construction. It also supplies a concrete counterexample to using depth alone as a universal difficulty measure: equal-depth, equal-work graphs can have different scheduling probabilities. For an unfamiliar controller, these profiles would be diagnostic evidence, not a unique identification of its internal mechanism.

Several limitations constrain the claims. The tasks are small and mechanical; the single execution tool omits natural-language instructions, tool discovery, and uncertain user goals. Immediate termination in the primary protocol magnifies compounding error and is a design choice, not a typical feature of software workflows. Even the follow-on maximum of sixteen operations does not represent long real-world workflows.

Active register count is not a lower bound on a controller's required memory. A controller could retain values modulo $b$, compress state, or precompute future answers, as the new controls explicitly demonstrate. The restricted-cache results measure chosen strategies, not minimum memory requirements. The hidden-resampling extension is one extreme missing-information mechanism, not a general model of partial observation, ambiguous feedback, or hidden user intent.

The graph families share the same state mechanics and are identical at the endpoint depths. The comparison is limited topology replication, not independent-domain generalization. Main contrasts include documented couplings, and the equal-weight grid is arbitrary. The large rollout count must not be confused with an equally large number of independent samples. All endpoint interactions are reported, but their bootstrap bands are descriptive and can degenerate at rare-event floors. No fitted universal difficulty model or out-of-domain predictive validation is supplied.

The next-use controller receives the same public graph as other controllers, but exploits future query knowledge and learns residues from successful trials. Its cache entry count does not include graph metadata, readiness bookkeeping, or controller code. Likewise, the residue-bit counts cover stored values, not total

| $b$ | Controller | Success (%) | Checks/call | Build entries | Lookups |
|---|---|---|---|---|---|
| 4 | Exact | 100.00 | 2.49 | 0 | 0.00 |
| 4 | Indexed | 100.00 | 0.00 | 32 | 8.00 |
| 4 | Scan-1 | 0.59 | 1.00 | 0 | 0.00 |
| 4 | Sham-check-4 | 0.59 | 4.00 | 0 | 0.00 |
| 4 | Fixed-check-4 | 100.00 | 4.00 | 0 | 0.00 |
| 8 | Exact | 100.00 | 4.52 | 0 | 0.00 |
| 8 | Indexed | 100.00 | 0.00 | 64 | 8.00 |
| 8 | Scan-1 | 0.00 | 1.00 | 0 | 0.00 |
| 8 | Sham-check-4 | 0.00 | 4.00 | 0 | 0.00 |
| 8 | Fixed-check-4 | 1.17 | 4.00 | 0 | 0.00 |

**Table 6: Selection ablation with full snapshots. Checks per call are the aggregate comparison count divided by execution calls. Build entries charge candidate-table preparation separately; lookup calls are separate operations, not equivalent CPU-time units. Early termination changes total comparisons, so only the fixed and redundant controls match comparison counts per decision.**

implementation memory. Preparation entries, lookups, and predicate checks are separate operation categories; none is a hardware-neutral measure of compute.

Language-model evidence is exploratory and limited to one Copilot-routed model pilot, a 16-episode half-fraction over two task seeds, and one small local-model factorial with a success floor. The Copilot pilot uses three paired task seeds per intervention cell, with its third seed added after inspection of the first two. The fractional factorial's last episode was completed after the first 15 outcomes had been inspected, though its prespecified condition was unchanged. These studies use one routed model and one quantized 0.5B model; none includes a reasoning-budget comparison, cross-provider comparison, or transfer to real tasks. They cannot establish which current model breaks first, whether more reasoning helps, or how the profiles predict real-task performance.

## 9 EXPLORATORY LANGUAGE-MODEL PILOT

At each decision, the model received the complete public task description, current observation, and interaction history, then returned one JSON action. The environment validated and executed the action; an invalid candidate or dependency choice ended the task. This is a serialized text-action interface, not native provider function calling. Copilot Auto selected the model, recorded as GPT-6 Luna throughout. The account did not expose a selectable Gemini or Qwen route or a reasoning-effort condition. This pilot therefore tests one model route, not providers or inference-budget scaling.

The pilot covers four paired low/high interventions in the chain family, with eight operations per task and three task seeds per cell. The first two seeds (16 episodes) were run before a third paired seed (eight episodes) was added; the latter was chosen after inspecting the initial outcomes. We therefore report only descriptive counts, without significance tests or generalization claims. Table 7 reports the first completed run for each unique factor–level–seed cell. The raw log contains 29 records, including five repeated/recovery records excluded from the 24-episode summaries. Success was 2/3 versus 3/3 for depth 1 versus 8, 2/3 versus 3/3 for branching 2 versus 8, 2/3 versus 2/3 for register width 1 versus 8, and 0/3 versus 2/3 for history versus snapshot feedback. These tiny counts are compatible with substantial sampling uncertainty; they do not establish that depth helps or that any factor has a stable model-level effect.

| Intervention | Level | Success | Mean completed | $n$ |
|---|---|---|---|---|
| Depth | 1 | 2/3 | 5.67 | 3 |
| Depth | 8 | 3/3 | 8.00 | 3 |
| Branching | 2 | 2/3 | 7.33 | 3 |
| Branching | 8 | 3/3 | 8.00 | 3 |
| Register width | 1 | 2/3 | 7.33 | 3 |
| Register width | 8 | 2/3 | 6.33 | 3 |
| Feedback | History | 0/3 | 5.00 | 3 |
| Feedback | Snapshot | 2/3 | 5.33 | 3 |

**Table 7: Exploratory Copilot-routed GPT-6 Luna pilot. Mean completed operations are out of eight. Each cell has three task seeds; no inferential claims are warranted.**

### 9.1 Two-seed fractional-factorial follow-up

We completed a second, separately frozen Copilot Auto study using the same text-action interface and GPT-6 Luna route. It covers eight conditions in a $2^{4-1}$ resolution-IV half fraction of depth (2 or 8), branching (2 or 8), register width (1 or 8), and history or snapshot feedback; every task requires eight operations. Two task seeds yield 16 episodes. The design is balanced and orthogonal for its main-effect contrasts, but main effects remain aliased with three-factor interactions, and two-factor interactions are mutually aliased. The two-seed sample is too small for reliable uncertainty estimates.

The model completed 8/16 workflows and 88/128 required operations (5.5 per task on average). Seven episodes ended in candidate-selection errors and one in a dependency error. The descriptive high-minus-low success differences were −25, +50, +25, and +50 percentage points for depth, branching, width, and snapshot feedback, respectively; mean completed-operation differences were −1.50, +1.50, +2.50, and +2.00. Table 8 summarizes these raw contrasts. The signs, including the apparent benefit of branching, are not a basis for mechanistic interpretation: there are only two seeds,

| Factor | Success difference | Operation difference | $n$ per level |
|---|---|---|---|
| Depth (8 vs. 2) | -25.0 pp | -1.50 | 8 |
| Branching (8 vs. 2) | +50.0 pp | +1.50 | 8 |
| Register width (8 vs. 1) | +25.0 pp | +2.50 | 8 |
| Snapshot vs. history | +50.0 pp | +2.00 | 8 |

**Table 8: Descriptive high-minus-low contrasts in the 16-episode Copilot-routed fractional factorial. Eight episodes contribute to each factor level; each of only two task seeds spans four conditions per level. Main effects may be aliased with three-factor interactions.**

possible three-factor aliasing, and a single model route. The last of 16 episodes was run after the first 15 outcomes had been inspected following an interruption; its condition and seed had already been specified, and the design was not changed. This exploratory result does not rescue the floor-level local-model factorial or establish a reproducible agent capability profile.

## 10 EXPLORATORY LOCAL-MODEL FACTORIAL

We ran a complete $2^5$ factorial of 256 tasks using the local, quantized Qwen2.5-0.5B-Instruct model [26]. The five controls were graph family (chains or barriers), dependency depth (2 or 8), candidate count (2 or 8), active-register width (1 or 8), and feedback format (history or current-state snapshot). Every task required eight operations. Eight task seeds were paired across all 32 cells. The protocol, factor grid, seeds, and analysis were fixed before this planned run; the protocol is local rather than externally preregistered.

The model received the complete public task specification, current observation, and interaction history, and returned one JSON action per decision. Invalid JSON or an invalid action ended the episode without a retry. Inference used llama.cpp server b12030 on CPU, Q4_K_M weights, temperature zero, and a 64-token output limit [27]. No hosted provider or paid inference was used. This is a single, small-model evaluation, not a comparison of providers or reasoning budgets. The separate runtime smoke test was excluded from the factorial and is retained in the artifact.

Across the planned tasks, the model completed 0/256episodes (0.000 success rate) and completed only 21 of 2048 required operations (0.08 of eight per task on average). Of 277 decisions, 255 (0.921) were strictly schema-valid JSON. The first-failure counts were 162 dependency errors, 54 candidate-selection errors, 18 invalid actions, 16 invalid action schemas, and 6 invalid JSON outputs. Median decision latency was 9.00 seconds on the local CPU.

Table 9 reports marginal high-minus-low success differences and seed-cluster bootstrap intervals. All are zero because every condition had zero completed workflows. These degenerate intervals are a floor effect, not evidence that the structural factors have no effect; the model was insufficiently capable on this interface to resolve their effects on the success endpoint. The run therefore documents a failed calibration of this model-task pairing, not a useful structural capability profile. It cannot establish generalization to larger or hosted models, and it does not test whether additional reasoning budget changes the profile.

## 11 NEXT EMPIRICAL TEST

The next study should freeze the interaction harness and task protocol before inference; use larger paired samples; and compare multiple models and providers. It should record model version, every prompt and action, valid action rate, completed operations, token usage, provider-reported cost, and latency. A reasoning-budget intervention must use a supported, explicit budget control and a total-usage-matched comparison. Retries and extra tool calls must remain separate interventions. An independent state-transition family and realistic transfer tasks are necessary before claiming generality. The present account has a 24-episode pilot and a 16-episode, two-seed fractional factorial using one Auto-routed model, plus a local small-model run at the success floor. Provider-level, cross-model, and reasoning-budget comparisons remain uncompleted.

## 12 CONCLUSION

Structural task controls are useful only when their invariants and couplings are explicit. Our fixed-work workflow instrument distinguishes scheduling, candidate-search, and retention restrictions with executable controls and exact calibration targets. The reference experiments show why a structural profile is more interpretable than its aggregate score, while also showing why depth, state width, and observation access cannot be treated as universally independent difficulty variables.

The local-model factorial is a failed calibration of the model-task interface, not evidence that the instrument yields a generalizable profile: one quantized 0.5B model completed none of 256 workflows, creating a floor on the success endpoint. The completed-operation and action-error records are useful diagnostics, but the zero marginal success contrasts cannot be interpreted as structural invariance. A second, 16-episode Copilot study shows nonzero contrasts, but two task seeds and fractional-factorial aliasing make them unstable descriptive observations, not evidence of dimension-specific effects. Claims about larger models, provider differences, reasoning-budget effects, or transfer to real workflows remain open.

*Reproducibility and research integrity.* The accompanying artifact contains both reference-task generators, the model-pilot, fractional-factorial, and local-model harnesses, 35 tests, protocols, episode traces, provider-usage records, structural audits, analysis scripts, generated tables, and source hashes. The 24 unique Copilot-routed model episodes use three paired seeds per intervention cell; the third seed was added after inspection of the first two, and five repeated/recovery records are excluded from their summaries. The Copilot fractional factorial has 16 episodes, two task seeds across eight planned half-fraction conditions, and 96 logged model calls; the final episode was completed after inspecting the first 15 outcomes, without changing the prespecified design. The separate local factorial uses one quantized Qwen2.5-0.5B model, eight paired seeds, and 256 planned episodes; it uses no paid inference. Copilot usage logs report 264 premium-request units and 25.653 billion nano-AIU across the recorded smoke and pilot runs;

| Contrast | Low success | High success | Difference (95% interval) |
|---|---|---|---|
| family (barriers − chains) | 0/128 (0.000) | 0/128 (0.000) | +0.0 pp (+0.0, +0.0) |
| depth (8 − 2) | 0/128 (0.000) | 0/128 (0.000) | +0.0 pp (+0.0, +0.0) |
| branching (8 − 2) | 0/128 (0.000) | 0/128 (0.000) | +0.0 pp (+0.0, +0.0) |
| width (8 − 1) | 0/128 (0.000) | 0/128 (0.000) | +0.0 pp (+0.0, +0.0) |
| observation (snapshot − history) | 0/128 (0.000) | 0/128 (0.000) | +0.0 pp (+0.0, +0.0) |

**Table 9: Exploratory local-model factorial. Each level contains 128 tasks. Differences are paired by task seed and averaged over the other factors; intervals resample the eight seed clusters.**

these are usage units, not dollars, and the account-wide quota is unavailable. There are no human-subject data or external-system actions. The artifact has not yet been deposited in a public archive.

## REFERENCES


[1] Kaijie Zhu, Jiaao Chen, Jindong Wang, Neil Zhenqiang Gong, Diyi Yang, and Xing Xie. 2024. DyVal: Dynamic Evaluation of Large Language Models for Reasoning Tasks. In *ICLR*. https://arxiv.org/abs/2309.17167.

[2] Parshin Shojaee, Iman Mirzadeh, Keivan Alizadeh, Maxwell Horton, Samy Bengio, and Mehrdad Farajtabar. 2025. The Illusion of Thinking: Understanding the Strengths and Limitations of Reasoning Models via the Lens of Problem Complexity. In *NeurIPS*. https://arxiv.org/abs/2506.06941.

[3] Xukai Wang et al. 2026. MorphoBench: A Benchmark with Difficulty Adaptive to Model Reasoning. In *Findings of ACL*. https://aclanthology.org/2026.findings-acl.1171/.

[4] Junlin Wang, Siddhartha Jain, Dejiao Zhang, Baishakhi Ray, Varun Kumar, and Ben Athiwaratkun. 2024. Reasoning in Token Economies: Budget-Aware Evaluation of LLM Reasoning Strategies. In *EMNLP*. https://aclanthology.org/2024.emnlp-main.1112/.

[5] Minghao Li et al. 2023. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *EMNLP*. https://arxiv.org/abs/2304.08244.

[6] Yujia Qin et al. 2023. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. *arXiv:2307.16789*. https://arxiv.org/abs/2307.16789.

[7] Harsh Trivedi et al. 2024. AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents. In *ACL*. https://arxiv.org/abs/2407.18901.

[8] Xiao Liu et al. 2024. AgentBench: Evaluating LLMs as Agents. In *ICLR*. https://arxiv.org/abs/2308.03688.

[9] Zixiang Liu et al. 2026. MCPAgentBench: A Real-world Task Benchmark for Evaluating LLM Agent MCP Tool Use. In *Proceedings of the Second Workshop for Research on Agent Language Models*. https://aclanthology.org/2026.realm-1.51/.

[10] Pengfei He et al. 2025. TRAJECT-Bench: A Trajectory-Aware Benchmark for Evaluating Agentic Tool Use. *arXiv:2510.04550*. https://arxiv.org/abs/2510.04550.

[11] Kunvar Thaman et al. 2026. Reward Hacking Benchmark: Measuring Exploits in LLM Agents with Tool Use. In *ICML*. https://arxiv.org/abs/2605.02964.

[12] Andrew Szot et al. 2026. Scaling Laws for Tool-Use Agents via Controllable Environment Synthesis. *arXiv:2610.37648*. https://arxiv.org/abs/2610.37648.

[13] Ruike Cao et al. 2026. MemCalib: Benchmarking and Optimizing Memory Use in LLM Agents. *arXiv:2609.24259*. https://arxiv.org/abs/2609.24259.

[14] Yongyi Wang, Lingfeng Li, Bozhou Chen, Ang Li, Hanyu Liu, Qirui Zheng, Xionghui Yang, and Wenxin Li. 2026. Synthetic POMDPs to Challenge Memory-Augmented RL: Memory Demand Structure Modeling. *Frontiers of Computer Science*. https://arxiv.org/abs/2508.04282.

[15] Bohan Tang, Huidong Liang, Keyue Jiang, and Xiaowen Dong. 2025. On the Importance of Task Complexity in Evaluating LLM-Based Multi-Agent Systems. *arXiv:2510.04311*. https://arxiv.org/abs/2510.04311.

[16] Ian Osband et al. 2019. Behaviour Suite for Reinforcement Learning. *arXiv:1908.03568*. https://arxiv.org/abs/1908.03568.

[17] Maxime Chevalier-Boisvert, Dzmitry Bahdanau, Salem Lahlou, Lucas Willems, Chitwan Saharia, Thien Huu Nguyen, and Yoshua Bengio. 2019. BabyAI: A Platform to Study the Sample Efficiency of Grounded Language Learning. In *ICLR*. https://arxiv.org/abs/1810.08272.

[18] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. $\tau$-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. *arXiv:2406.12045*. https://arxiv.org/abs/2406.12045.

[19] Thomas Kwa et al. 2025. Measuring AI Ability to Complete Long Software Tasks. In *NeurIPS*. https://arxiv.org/abs/2503.14499.

[20] Pedro Gimenes, Zeyu Cao, Jeffrey Wong, and Yiren Zhao. 2025. ARIES: Autonomous Reasoning with LLMs on Interactive Thought Graph Environments. *arXiv:2502.21208*. https://arxiv.org/abs/2502.21208.

[21] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. 1998. Planning and Acting in Partially Observable Stochastic Domains. *Artificial Intelligence* 101, 1–2, 99–134. https://doi.org/10.1016/S0004-3702(98)00023-X.

[22] Rory Lipkis and Adrian Agogino. 2025. Failure Analysis of Autonomous Systems with RL-Guided MCMC Sampling. In *Adaptive and Learning Agents Workshop*. https://ala-workshop.github.io/papers/ALA2025_paper_45.pdf.

[23] Kevin A. Wang, Jerry Xia, Stephen Chung, and Amy Greenwald. 2025. Dynamic Thinker: Optimizing Decision-Time Planning with Costly Compute. In *Adaptive and Learning Agents Workshop*. https://ala-workshop.github.io/papers/ALA2025_paper_44.pdf.

[24] Narjes Nourzad, Jared Coleman, Zhongyuan Zhao, Bhaskar Krishnamachari, Gunjan Verma, and Santiago Segarra. 2025. Actor-Twin Framework for Task Graph Scheduling. In *Adaptive and Learning Agents Workshop*. https://ala-workshop.github.io/papers/ALA2025_paper_17.pdf.

[25] Augusto Leal, Mateus Begnini Melchiades, and Gabriel de Oliveira Ramos. 2025. A Flexible Approach to Deliberation Cost in the Option-Critic Architecture. In *Adaptive and Learning Agents Workshop*. https://ala-workshop.github.io/papers/ALA2025_paper_28.pdf.

[26] An Yang et al. 2024. Qwen2.5 Technical Report. *arXiv:2412.15115*. https://arxiv.org/abs/2412.15115.

[27] ggml-org. 2026. llama.cpp, release b12030. https://github.com/ggml-org/llama.cpp/releases/tag/b12030.